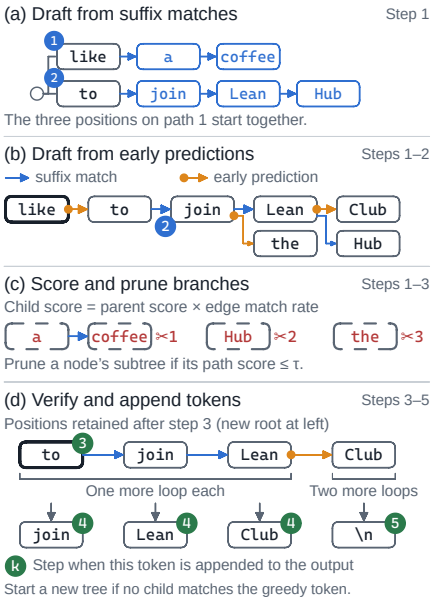


Figure 2. **LoopFill decoding with three loops per position.** (a) A suffix match at **[like]** proposes **[a] [coffee]**. (b) Both early-loop predictions (orange edges) and suffix-tree matches (blue edges) add tokens to the same draft tree. The first loop at **[like]** adds **[to]**, and a suffix match supplies the next edge from **[to]** to **[join]**. (c) Pruning **[a]** at step 1 also removes **[coffee]**. (d) After the decoder verifies **[to]** at step 3, the retained positions predict the tokens shown below them. Green badges give the step when each output token is appended.

3. LoopFill Decoding

LoopFill builds a single *draft tree* from two sources. As in SuffixDecoding (Oliaro et al., 2025), suffix matches supply continuations that occurred in earlier text. As in LoopSpec (Cho et al., 2026), the model can propose a next token before the current position finishes all T loops. A suffix match can extend a token proposed by an early loop, and an early prediction can extend a token retrieved from the suffix tree.

Each additional draft position consumes model computation on every loop it runs. When many requests share a device, spending that computation on branches that will be discarded can lower output throughput. For candidates from either source, we estimate how often they match the full-depth greedy next token on calibration prompts. LoopFill combines these estimates along each path and processes a draft node only while its path score exceeds a threshold. Only the full-depth prediction at the verified prefix determines which token the decoder appends.

Figure 2 follows the example in Figure 1. With $T = 3$, LoopFill produces the five output tokens in five loop steps. Greedy decoding takes fifteen.

Growing the draft tree. The *suffix tree* indexes the prompt, the verified output so far, and outputs from earlier requests. A query matches a suffix of a given prefix against the index and returns the continuations that followed the match. The *draft tree* stores the candidates that LoopFill will process for the current request. Its root represents the verified prefix x. A node u represents x followed by the draft tokens on the path to that node. A child ua extends u by one candidate token a.

Each node processes the last token of its prefix to predict the token *after* that prefix. Attention at the node uses only the verified prefix and the tokens on its own path. In Figure 2(a), a suffix match lets **[a]** and **[coffee]** start their first loop alongside **[like]**. Each step advances every unfinished position by one loop. The decoder reads the next-token distribution at u after d_1 loops and again if u reaches d_2 loops ($d_1 < d_2 < T$). We call each readout a *shallow prediction*. The candidates for the next position are the top eight tokens in that distribution together with the tokens labeling the existing children of u. When the decoder admits a new child ua from a shallow prediction at u, it queries the suffix index with the prefix ending at ua. Nodes admitted during one step can start their first loop in the next step.

Figure 2(b) shows how the two sources extend the same branch. The first loop at **[like]** proposes **[to]**. Querying the suffix tree with the prefix ending in **[to]** adds the path **[join] [Lean] [Hub]**. All four positions start in step 2. The first loop at **[Lean]** then proposes **[Club]** as another child of **[Lean]**.

Calibrating and pruning drafts. Computing a node is useful for the output only if every token on its path agrees with greedy decoding. For each edge from u to ua, we estimate the probability $\hat{q}(a \mid u)$ that the greedy next token after u is a, conditional on u being on the greedy path. Before decoding, we generate greedy outputs for a set of calibration prompts. At prefixes on those outputs we group candidates by the features below and record the fraction in each group that match the full-depth greedy next token. For suffix candidates we collect these observations at the prefixes where the decoder actually queries the suffix index.

For a suffix proposal, the features m record the suffix-match length, the relative frequency of a among continuations, whether u has other children, and the depth of u in the draft tree. Let b indicate whether the suffix tree proposed a. Let r_i and p_i be the rank and probability of a in the shallow prediction at loop d_i. A token outside the top eight receives a reserved rank and $p_i = 0$. The decoder updates the edge estimate using fixed match-rate tables M.

$$\hat{q}(a \mid u) \leftarrow \begin{cases} M_{\text{suf}}(m) & \text{on a suffix proposal,} \\ M_1(r_1, p_1, b) & \text{after } d_1 \text{ loops,} \\ M_2(r_1, r_2, p_2, b) & \text{after } d_2 \text{ loops.} \end{cases}$$

Each update replaces the previous estimate for that edge. The root score is $s(x) = 1$, and a child's path score is $s(ua) = s(u)\,\hat{q}(a \mid u)$. For example, edge estimates of 0.8 for **[to]** after the root and 0.5 for **[join]** after **[to]** give the path **[to] [join]** a score of 0.4.

The decoder keeps a draft node only while its path score exceeds τ. When an edge estimate changes, the decoder recomputes the scores of all paths through that edge. When a node's score falls to τ or below, the decoder removes that node and its descendants. In Figure 2(c), the score for **[a]** falls to τ or below at step 1, so pruning **[a]** also removes its child **[coffee]**.

To select τ on development prompts, we count both the sequential loop steps per output token and the number of positions processed per output token, counting a position once for each loop it runs. A lower threshold can save steps by starting correct positions earlier. A higher threshold can reduce the work spent on incorrect branches. We use $\tau = 0.01$ for the low-load setting and $\tau = 0.1$ when limiting draft computation takes priority.

Verifying tokens and reusing completed loops. When the root for x finishes all T loops, the decoder takes the greedy next token a at that root, appends a to the output, and adds a to the suffix index. If the root has a child xa, the decoder makes that child the new root with score one, keeps the child's subtree and its completed loops, and discards the old root's other branches. From this new root the decoder recomputes path scores and queries the suffix index for continuations of xa. If no child matches a, the decoder starts a new draft tree at xa. Every emitted token is therefore the full-depth greedy prediction at its verified prefix.

A new root that has already finished all T loops can supply the next output token in the same step. In Figure 2(d), the final prediction at the position processing **[like]** is **[to]** in step 3. By then **[to]**, **[join]**, and **[Lean]** have each completed two loops. In step 4 those positions finish together, and the decoder appends their predictions **[join]**, **[Lean]**, and **[Club]** in order. In step 5 the position processing **[Club]** finishes its last loop and predicts **[\n]**. Algorithm 1 summarizes one loop step.

Algorithm 1. LoopFill decoding

Start. Create a draft tree with verified prefix x as its root and $s(x) = 1$. Add suffix continuations one token at a time while their path scores exceed τ.

Repeat for each loop step
  Advance every unfinished node by one loop.
  While the root for x has completed T loops
    Let a be the root's greedy next token. Append a to the output and add a to the suffix index. Stop if the output is complete.
    Set $x \leftarrow xa$. If the tree has no node for the updated prefix x, return to **Start**.
    Make that node the root with $s(x) = 1$. Retain its subtree and completed loops, recompute path scores from x, and add suffix drafts from x.
  For each surviving node u that just completed d_1 or d_2 loops
    Score the existing children of u and the top eight next tokens in its shallow prediction. Keep a candidate ua only if $s(ua) > \tau$. Otherwise discard ua and any descendants.
    Add suffix drafts from each newly added child.

References

Oliaro, G., Jia, Z., Campos, D., and Qiao, A. SuffixDecoding: Extreme speculative decoding for emerging AI applications. In *Advances in Neural Information Processing Systems*, 2025.

Cho, S., Cui, L., Kim, S., Han, D., and Han, I. LoopSpec: Pipelined self-speculative decoding for looped transformers. *arXiv:2609.17184*, 2026.